

Programación III — Propiedades de Algoritmos

Análisis de Sistemas Canónicos de Monedas y la Propiedad de Elección Golosa

Para profundizar en el comportamiento del **Algoritmo Greedy**, es fundamental comprender por qué requiere que el conjunto de datos cumpla ciertas **propiedades estructurales** y qué ocurre matemáticamente cuando un sistema de monedas viola dichas condiciones.

1. ¿Por qué se exige una Propiedad Estructural?

El algoritmo Greedy opera bajo la premisa de que tomar la decisión *localmente óptima* en cada paso conduce de forma inevitable al *óptimo global*. Para que esta premisa se cumpla en el problema del cambio de monedas, el sistema debe ser **canónico**.

Propiedad de Dominancia (Regla del Doble / Divisibilidad):

En un sistema de monedas bien diseñado, cada moneda M_k debe ser al menos el doble de la inmediata inferior ($M_k \geq 2 \cdot M_{k-1}$) o mantener una relación estricta de divisibilidad. Esto garantiza que tomar una moneda de mayor valor nunca "prive" al algoritmo de una combinación más eficiente formada por monedas menores.

2. Análisis del Fallo en el Sistema Imperial Británico

Considerando el residuo de **48 peniques** dentro del sistema británico antiguo, analizamos las denominaciones clave:

[... , Media Corona (30), Florín (24), Chelín (12), 6p, 3p, 1p]

A. Violación de la condición estructural

- La moneda anterior a **30** es la de **24**.
- Para cumplir la regla de dominancia, la moneda superior a 24 debería valer al menos **48** ($24 \times 2 = 48$).
- Sin embargo, existe una moneda intermedia de **30**. Como $30 < 2 \times 24$, la moneda de 30 "compite" con dos de 24, pero no es lo suficientemente grande como para absorberlas o superarlas en eficiencia.

La "Trampa" de la Elección Golosa:

Al intentar pagar **48 peniques**, la moneda de 30 es *demasiado grande* para permitir el uso de dos monedas de 24 (ya que $48 - 30 = 18 < 24$), pero *no lo suficientemente grande* para resolver el pago por sí sola. Al tomar la de 30, el algoritmo "bloquea" la solución óptima de 2 monedas y se ve obligado a usar 3 ($30 + 12 + 6$).

3. Cuadro Comparativo de Sistemas

Característica	Sistema Canónico (Ej. Actual)	Sistema No Canónico (Ej. Imperial)
Denominaciones	1, 2, 5, 10, 20, 50, 100	1, 3, 6, 12, 24 , 30 , 60, 240
Relación $M_k \geq 2 \cdot M_{k-1}$	Sí cumple (crecimiento adecuado)	NO cumple ($30 < 2 \cdot 24$)
Resultado Greedy (para 48p)	Garantiza el óptimo global	Subóptimo: usa 3 monedas (\$30+12+6\$)
Resultado Óptimo Real	Mismo resultado que Greedy	Óptimo real: usa 2 monedas (\$24+24\$)
Técnica Requerida	Algoritmo Greedy ($\Theta(v)$)	Programación Dinámica ($\Theta(v \cdot k)$)

4. Conclusión Técnica

Cuando un sistema de monedas es **no canónico**, la estructura de subproblemas óptimos se solapa de forma compleja. En estos escenarios, la propiedad de elección golosa deja de ser válida y es técnicamente obligatorio recurrir a la **Programación Dinámica** para evaluar todas las combinaciones mediante la ecuación de recurrencia:

$$C(v) = 1 + \min_{\{i: d_i \leq v\}} \{ C(v - d_i) \}$$